

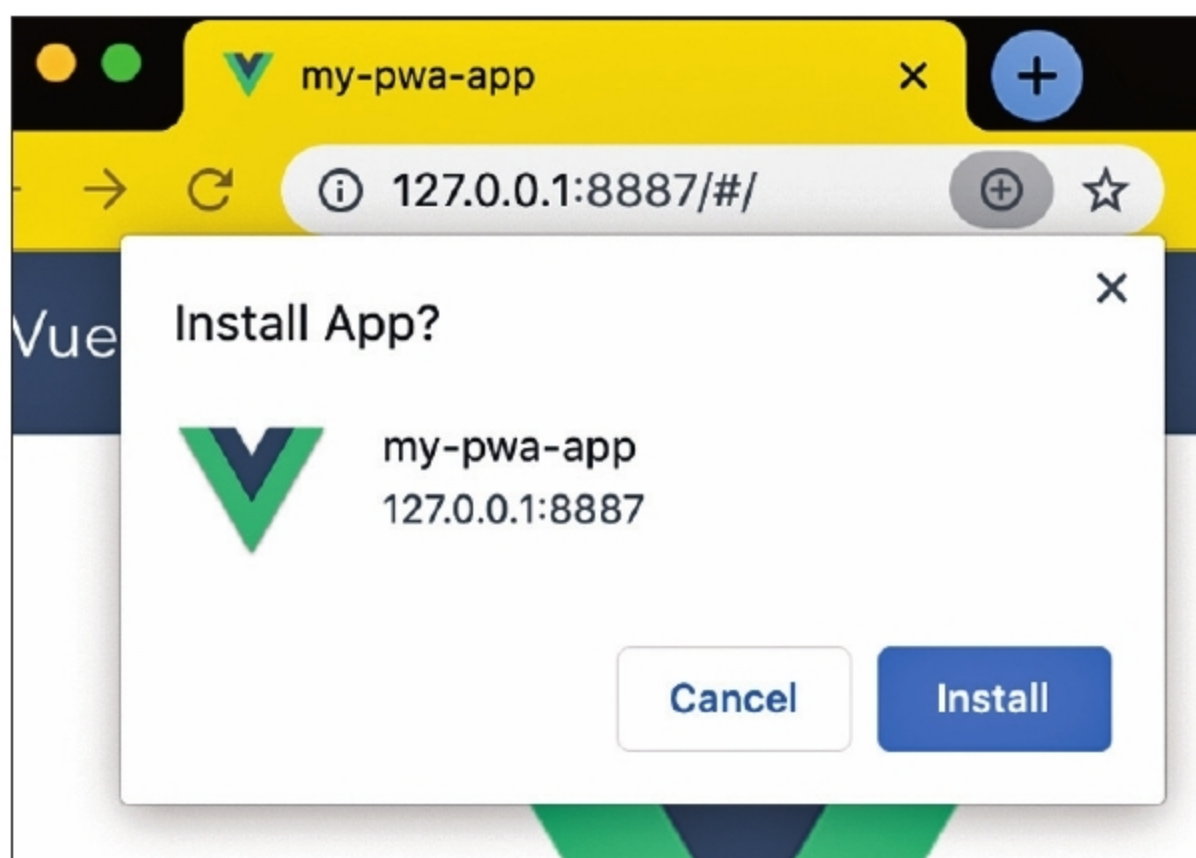


Figure 6: Install your PWA in Chrome

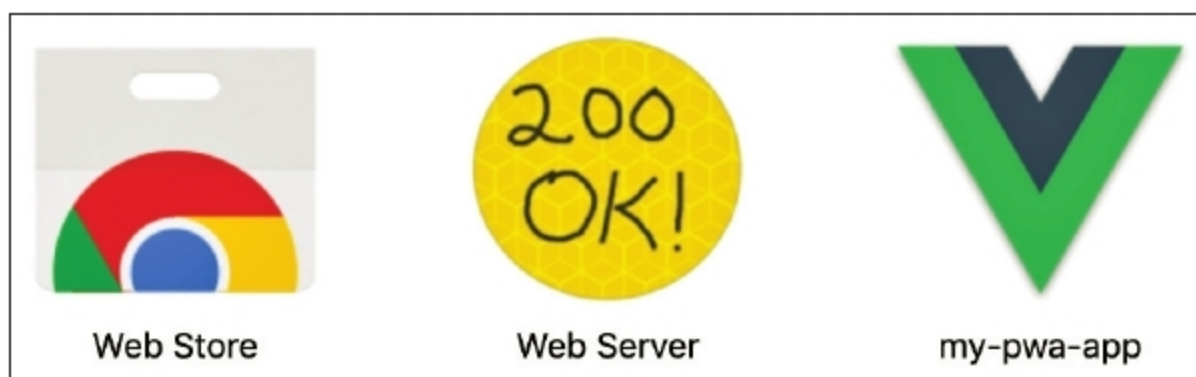


Figure 7: Your PWA in Chrome apps

To be able to run the PWA on a Web server, we first need to build it in order to create a production ready *dist* folder within our project directory. To build the app, execute the following command:

```
npm run build
```

Now, if you type *ls* in your terminal, you will be able to see a new *dist* folder created, as shown in Figure 4.

Next, open the 200 OK app in your Chrome browser. Click 'Choose folder' and add the path to the *dist* folder we just created in the project directory, as shown in Figure 5.

Click on the Web server URL and that's it! Your PWA is now running on a Web server.

Now you will be able to see a '+' button next to your app URL. If

you click on it, you will be given an option to install *my-pwa-app*, as shown in Figure 6. Alternatively, you can find the option in the top right menu as well.

After installing your app, you can access it from *chrome://apps*, as shown in Figure 7. In a mobile phone, you can save your app directly to your home screen to run it natively.

The key elements of a PWA

There are two major files in your project directory that turn your app into a PWA. The first one is the *manifest.json*. This file provides all the necessary information about your app to the browser, such as the name of your app, the location of the app icons, etc.

The next most important file is the service worker that acts as an intermediary between your browser and the network connection, caching important data and improving the app's performance. A service worker enables a user to access the cached content of the app when the user is offline, hence making the app mostly (if not completely) native to the user's device.

The 'Pomodoro clock': A simple PWA example using Vue.js

The best way to understand the benefits of a PWA is by creating a project or app of your own. Pomodoro [https://en.wikipedia.org/wiki/Pomodoro_Technique] is a famous time management technique using which you work in intervals of 25 minutes, followed by short breaks for increased productivity. As a demonstration, I created a simple Pomodoro clock PWA using Vue.js. Discussing the code for the entire app is out of scope for this article so I uploaded the repository on GitHub [<https://github.com/shah78677/Pomodoro-Clock>]. I encourage you to fork the repository and play around with it to see what functionalities you can add to a PWA. **END** 🐧

References

- [1] <https://nodejs.org/en/download/>
- [2] https://en.wikipedia.org/wiki/Pomodoro_Technique
- [3] <https://github.com/shah78677/Pomodoro-Clock>

By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his works at codelatte.site.